PYTHON FILE HANDLING

WHAT IS A FILE?

- Named collection of related data
- Stored as a single unit on a storage medium.
- Os treats it as one object with:
  1. A filename
  2. Extension
  3. Metadata
  4. Contents

Key properties of files

1. Persistent
2. Addressable by name/path
3. Managed by the file system

FILE OPEN

- Open a File on the Server
- Assume we have the following file, located in the same folder as Python:
- To open the file, use the built-in open() function.
- The open() function returns a file object, which has a read() method for reading the content of the file
- If the file is located in a different location, you will have to specify the file path,

OPENING A FILE

- Opening is done with the **open()**
- Syntax : **file-object = open(“File-name / File-path”, “access-mode)**

- Example:

```
file = open('example.txt', 'r')
content = file.read()
print(content)
file.close()
```

ACCESS-MODES

Mode	Type	Description / Use	Pointer Position	File Created?	Overwrites?	Error Conditions
r	Read	Read-only mode	Beginning	❌ No	❌ No	❌ Raises error if file does NOT exist
w	Write	Write-only mode	Beginning	✅ Yes	✅ Yes (clears file)	❌ No error if file exists
a	Append	Add data to end	End of file	✅ Yes	❌ No	❌ No error if exists
r+	Read + Write	Read and write	Beginning	❌ No	❌ No	❌ Error if file does NOT exist
w+	Write + Read	Write then read	Beginning	✅ Yes	✅ Yes	❌ No error if exists
a+	Append + Read	Read & append at end	End of file	✅ Yes	❌ No	❌ No error if exists

ACCESS-MODES

Mode	Type	Description / Use	Pointer Position	File Created?	Overwrites?	Error Conditions
x	Exclusive Create	Creates new file only	Beginning	✅ Yes	❌ No	✅ Error if file already exists
rb	Read Binary	Read binary data	Beginning	❌ No	❌ No	❌ Error if file does NOT exist
wb	Write Binary	Write binary data	Beginning	✅ Yes	✅ Yes	❌ No error if exists
ab	Append Binary	Append binary data	End of file	✅ Yes	❌ No	❌ No error if exists
rb+ / r+b	Read + Write Binary	Read + write binary	Beginning	❌ No	❌ No	❌ Error if file does NOT exist
wb+ / w+b	Write + Read Binary	Write + read binary	Beginning	✅ Yes	✅ Yes	❌ No error if exists
ab+ / a+b	Append + Read Binary	Read + append binary	End of file	✅ Yes	❌ No	❌ No error if exists

FILE READ

- By default, the read() method returns the whole text, but you can also specify how many characters you want to return
- Read Lines
- You can return one line by using the readline() method
- By calling readline() two times, you can read the two first lines
- By looping through the lines of the file, you can read the whole file, line by line
- Close Files
- Syntax : fileobj.read(<count>)

FILE WRITE

- Write to an Existing File
- To write to an existing file, you must add a parameter to the open() function
- "a" - Append - will append to the end of the file
- "w" - Write - will overwrite any existing content
- the "w" method will overwrite the entire file.
- Create a New File
- To create a new file in Python, use the open() method, with one of the following parameters
- "x" - Create - will create a file, returns an error if the file exist
- "a" - Append - will create a file if the specified file does not exist
- "w" - Write - will create a file if the specified file does not exist

DELETE FILE

- Example:

```
import os
os.remove("three.txt")
os.removedirs("hai")
os.removedirs("hello")
```

- To delete a file, you must import the OS module, and run its os.remove() function
- Check if File exist:
- To avoid getting an error, you might want to check if the file exists before you try to delete it
- Delete Folder
- To delete an entire folder, use the os.rmdir() method

OS MODULE IN FILE OPERATIONS

used to interact with the **operating system.**
allows you to perform **file and directory operations**

Operation	Function	Example
Check existence	os.path.exists()	os.path.exists("a.txt")
Rename file	os.rename()	os.rename("a.txt","b.txt")
Delete file	os.remove()	os.remove("a.txt")
Create folder	os.mkdir()	os.mkdir("data")
Create nested folders	os.makedirs()	os.makedirs("a/b/c")
Remove empty folder	os.rmdir()	os.rmdir("data")
Remove folder (all)	shutil.rmtree()	shutil.rmtree("a")
List directory	os.listdir()	os.listdir(".")
Current directory	os.getcwd()	os.getcwd()
Change directory	os.chdir()	os.chdir("path")
Check file	os.path.isfile()	os.path.isfile("a.txt")
Check folder	os.path.isdir()	os.path.isdir("folder")

CLOSE FILE

- Used to **close an opened file** and free the resources associated with it.
- **Syntax :** file_object.close()
- Example:

```
f = open("example.txt", "r")
print(f.read())
f.close()
#Checking if File Is Closed
Print( f.closed) # True
```

What Happens If You Don't Close?

- Data may not be saved fully
- System may hit "too many open files" error
- File may remain locked
- Memory leakage

THE WITH STATEMENT

- used to **open a file and automatically close it** after the block of code is executed.

Feature	Explanation
Auto-closes file	File is closed automatically after the block ends.
Prevents data loss	Ensures buffered data is written.
Prevents file corruption	File closes even if an error occurs inside the block.
Cleaner code	No need to call close() manually.

PATHLIB

- Modern Python module used for **working with file paths** and performing **file operations** in an object-oriented way.
- It provides a Path class that represents filesystem paths
- Offers methods to perform common operations

Why use pathlib ?

Feature	Benefit
Object-oriented	No string paths, simpler path handling
Cross-platform	Works on Windows, macOS, Linux the same way
Easy path joining	No need for / or \\
Built-in file methods	Read, write, exists, rename, delete etc.

DIFFERENCE B/W PATHLIB AND OS MODULE

Feature	pathlib	os / os.path
Programming Style	Object-oriented	Function-based (procedural)
Path Type	Uses Path objects	Uses strings
Path Joining	Uses / operator → Path("a") / "b.txt"	Uses os.path.join("a", "b.txt")
Cross-Platform Handling	Automatically handles OS-specific separators	Must manually manage "/" or "\" or use os.path.join
Read/Write Files	Built-in methods → read_text(), write_text()	Uses open() function
Path Properties	Easy access: p.name, p.stem, p.suffix	Must use functions → os.path.basename(path)
Directory Iteration	iterdir(), glob(), rglob()	os.listdir(), glob.glob()
Creating Directories	Path.mkdir()	os.mkdir() / os.makedirs()
Delete File	unlink()	os.remove()
Check File/Folder	is_file(), is_dir()	os.path.isfile(), os.path.isdir()
Absolute Path	resolve()	os.path.abspath()
Easier to Read and Understand	Yes	Less readable for complex paths
Introduced in	Python 3.4	Python 2+ (older)